

Week 2a: Informal Proofs and Reductions

Nadav Kohen

June 29, 2025

Now that we have introduced some of the basic constructions and vocabulary from public key cryptography, our goal this week is to begin our path towards proving that protocols are secure. This turns out to require two distinct skills, so we have split this week into two parts. In this first part (Week 2a), we will practice writing *informal* mathematical proofs, focusing on the single most important proof technique in cryptography: *proof by reduction*. In the second part (Week 2b), we will develop just enough *probability* theory to state security definitions precisely and to verify that our informal arguments actually hold up. The two parts are designed to be read in order, and together they set us up to study the security of Schnorr signatures next week.

Before we begin, a word on what we mean by proof. A proof is a convincing argument that some statement must be true. It should be convincing in the sense that it leaves the reader no room to doubt your conclusions if they accept your premises. Of course, this definition of convincing varies by reader; writing for a beginner audience requires far more justification and detail than writing for experts usually does. For the purposes of this course, I generally recommend trying to write in such a way that would remove all doubt from a reader who is a little less skilled and a little less knowledgeable than you were before you worked on the problem. Ideally you could find your proofs easily readable if you review them a year from now. All of this said, I am not asking you to write formal or fully rigorous

proofs, and I am expecting quite a bit of informal reasoning and maybe even some hand-waving. This will be forced upon you by the fact that I won't provide rigorous definitions of all concepts yet, opting instead to start with intuitive notions. Remember, no one is able to write fully rigorous proofs without practice, and these exercises are intended as practice so don't worry too much about perfection. Err instead on the side of getting feedback early and often on your proof attempts.

Proof by contrapositive

Perhaps the most prevalent proof technique in cryptography is proof by reduction, which is itself a special case of proof by contrapositive. In logic, an implication $P \Rightarrow Q$ is true if Q must be true whenever P is true. The contrapositive of an implication $P \Rightarrow Q$ is the equivalent implication $\neg Q \Rightarrow \neg P$, where $\neg$ denotes negation (i.e., $\neg Q$ is read “not Q ”). These implications are equivalent because each of them is true if and only if P is false or Q is true. Hence, if one is trying to argue some conclusion, Q , from some assumption, P , argument by contrapositive is the method of assuming that the conclusion is false and arguing that this forces the assumption to be false.

There is a closely related concept called proof by contradiction, in which we prove $P \Rightarrow Q$ by assuming Q is false while assuming P is true and arriving at a contradiction (such as a statement like $1 = 0$). This proves Q must be true as otherwise a contradiction arises. Oftentimes a proof by contrapositive is written like a proof by contradiction, which is fine, but I personally believe that if you can use contrapositive then that is superior to contradiction. The reason I believe this despite them being equivalent is that *if* you make a mistake in your argument, then your proof by contrapositive is likely to fail (since it is built out of a series of implications). On the other hand, if you make a mistake during a proof by contradiction you might be “rewarded” for your misstep as it becomes even easier to find a contradiction

from flawed reasoning.

It is worth pausing on a small, non-cryptographic example to see the shape of argument by contrapositive before we apply it to anything hard. Suppose we want to prove the implication “if n^2 is even, then n is even,” where n is a whole number. Arguing this directly (that is, starting at the premise that n^2 is even and using various facts to imply the conclusion that n is even) is awkward, but the contrapositive is easy: “if n is *not* even (i.e., n is odd), then n^2 is *not* even (i.e., n^2 is odd).” To see this, write an odd number as $n = 2k + 1$; then $n^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$, which is odd. Having proven the contrapositive, we have proven the original statement. Notice the move we made: rather than starting from the hypothesis we cared about, we started by *assuming the conclusion was false* and showed that this is impossible unless the hypothesis was also false.

Exercise 1. *Warm-up. Let n be a whole number. Using proof by contrapositive, show that if n^2 is a multiple of 3, then n is a multiple of 3. State clearly what P and Q are, what their negations are, and where in your argument you use the assumption that the conclusion is false. (Hint: every whole number can be written as $3k$, $3k + 1$, or $3k + 2$.) Optional fun extension: Can you generalize the example above and the result and proof of this exercise?*

Proof by reduction

In cryptography, we very often want to show that if we assume that problem X is hard to solve, then another problem, Y , is also hard to solve. Written as an implication, this is “(X is hard) $\Rightarrow$ (Y is hard).” Its contrapositive is “(Y is *not* hard) $\Rightarrow$ (X is *not* hard),” or in plainer terms, “if Y is easy, then X is easy.” A proof by reduction is exactly a proof of this contrapositive: we assume we are handed an efficient solution to Y as a black box, and we use it as a subroutine to build an efficient solution to X .

A useful way to picture this: imagine someone sells you a magic box that efficiently solves problem Y . A reduction is a recipe that wires that box up — possibly transforming its inputs and outputs — so that the whole contraption efficiently solves problem X . If such a recipe exists, then “ Y is easy” really does imply “ X is easy,” and so (taking the contrapositive) “ X is hard” implies “ Y is hard.” The art of a reduction is figuring out how to feed your Y -box the right inputs and interpret its outputs to extract a solution to X .

Let’s take a moment to understand this through a concrete example.

Recall that the Discrete Log (DL) assumption for a group $\mathbb{G}$ is that it is hard to compute x given g^x as input, where g is the generator for $\mathbb{G}$. A related assumption, known as the One More Discrete Log (OMDL) assumption for $\mathbb{G}$, states that if a program is given some (positive) number, k , of random group elements, $g^{x_1}, g^{x_2}, \dots, g^{x_k}$, and it is able to ask for and receive discrete logarithms for any $k-1$ group elements, then computing all of the values $x_1, x_2, \dots, x_k$ is still hard. We can use reduction to see that if the OMDL assumption holds for a group, then the DL assumption holds. Specifically, this proof by reduction requires us to show that if the DL problem is easy to solve, then it will be easy to solve the OMDL problem. But of course, if we can compute discrete logarithms in a group with non-negligible probability (i.e., the DL problem is easy), then we can trivially solve the OMDL problem by asking for the discrete logs for $g^{x_1}, \dots, g^{x_{k-1}}$ and then using our DL-solver to directly compute the k th discrete log with non-negligible probability.

Let us name the parts of that argument so the pattern is visible, since you will reproduce it many times. Here X is the OMDL problem (the one we assume is hard) and Y is the DL problem (the one we assume, for contradiction, is easy). The “magic box” is the DL-solver. The reduction is: forward the first $k-1$ challenges to the discrete-log oracle we are allowed to query, run the DL-solver on the final challenge, and output everything. Because the box

succeeds with non-negligible probability, so does our contraption — which is what it means for OMDL to be easy.

Before tackling a full cryptographic reduction, it helps to do one where the box and the wiring are about as simple as possible.

Exercise 2. *Warm-up reduction. Recall the Diffie-Hellman setup, where two parties have public keys $X = g^x$ and $Y = g^y$ and their shared secret is $Z = g^{x \cdot y}$. Suppose someone hands you an efficient “magic box” that solves the discrete log problem: given any group element g^a it returns a . Using this box as a subroutine, describe an efficient procedure that, given only $X = g^x$ and $Y = g^y$ (but not x or y), computes the Diffie-Hellman shared secret $Z = g^{x \cdot y}$. Then state, in one sentence, which implication between assumptions you have just informally proven via its contrapositive.*

We are now ready for a reduction that matters for the rest of the course.

Exercise 3. *Let $\mathbb{G}$ be an abelian group with an element, $g \in \mathbb{G}$, of prime order n . A Schnorr signature of a message, m , by a private key, $x \in \mathbb{Z}/n\mathbb{Z}$, is a pair (R, s) such that $s = k + H(R, m) \cdot x \in \mathbb{Z}/n\mathbb{Z}$ and $R = g^k \in \mathbb{G}$ is the nonce (k is a random element of $\mathbb{Z}/n\mathbb{Z}$ generated for this signature). Intuitively, this works because s commits to the message using a hash, multiplies by the private key, but it does not reveal the private key because of the random blinding value k . A Schnorr signature can be verified by checking the equation $g^s == R \cdot X^{H(R, m)}$, where $X = g^x$ is the signer’s public key.*

Use proof by reduction to show that if we assume that the discrete log problem is hard in $\mathbb{G}$, then we can conclude that it is hard to forge two Schnorr signatures for the same nonce and key, that is, if we are given a public key $X = g^x$ and nonce R (without being given x or k) it is hard to compute two different messages m_1 and m_2 and values s_1 and s_2 such that (R, s_1) and (R, s_2) are valid signatures for X of m_1 and m_2 , respectively.

This will later be an important part of the proof of the stronger statement that if the

discrete log problem is hard, then Schnorr signatures are unforgeable in a much stronger sense.

Attack Games: making “hard” and “easy” more precise

In the above exercises, we are working with an informal notion of what it means for something to be “hard” and for something to be “easy,” and we will now want to make this more precise. (We will only make it *fully* precise once we have probability in Week 2b and a notion of negligible functions in Week 3; for now we are formalizing the *structure* of these claims.)

In cryptographic assumption definitions, we make these kinds of statements precise and functional by formalizing them as attack games. In an attack game, we model the adversary as a program that interacts with another program, the challenger (aka the environment, aka the context). In this case, we have to define a very simple game where the adversary is challenged to compute the discrete log of a group element (where g is a fixed generator), as depicted in the attack game $\text{DL}^{\mathcal{A}}(\lambda)$ in Figure 1. Note that this game is parameterized by λ , which is the key size. We take this opportunity to introduce two other common cryptographic assumptions relating to last week’s Diffie-Hellman key exchange. Recall that in the Diffie-Hellman key exchange, each party has a public key, $X = g^x$ and $Y = g^y$, and then their shared secret is $Z = g^{x \cdot y}$. The Computational Diffie-Hellman (CDH) assumption is that it is “hard” to compute Z given only X and Y , this is shown in the attack game $\text{CDH}^{\mathcal{A}}(\lambda)$ in Figure 1. Lastly, there is a variation of the CDH assumption called the Decisional Diffie-Hellman (DDH) assumption, which states that it is “hard” to distinguish between a Diffie-Hellman shared secret and a truly random group element (given access to the public keys X and Y); this is made precise in the attack game $\text{DDH}^{\mathcal{A}}(\lambda)$ in Figure 1.

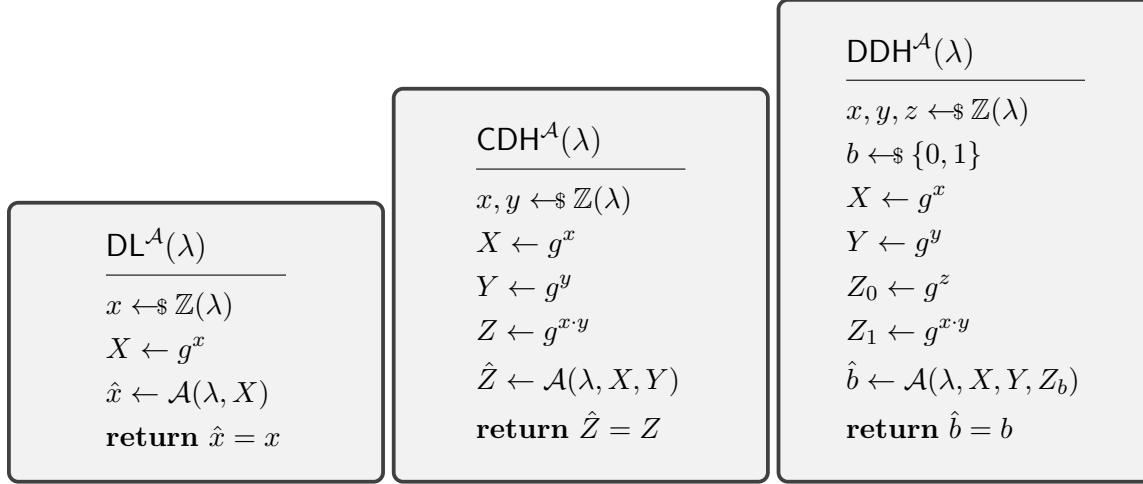


Figure 1: Attack games for the DL, CDH, and DDH cryptographic assumptions. ($\mathbb{Z}(\lambda)$ is $\mathbb{Z}_p$ for a prime, $p > 2^\lambda$, which makes λ the key size, g is a fixed generator for the group $\mathbb{G}(\lambda)$ of size p , and the symbol $\$$ means chosen uniformly at random).

For each of these games, we define the advantage an adversary program, $\mathcal{A}$, has as

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{dl}}(\lambda) &:= \Pr [\text{DL}^{\mathcal{A}}(\lambda) = \text{true}] , \\ \text{Adv}_{\mathcal{A}}^{\text{cdh}}(\lambda) &:= \Pr [\text{CDH}^{\mathcal{A}}(\lambda) = \text{true}] , \\ \text{Adv}_{\mathcal{A}}^{\text{ddh}}(\lambda) &:= \Pr [\text{DDH}^{\mathcal{A}}(\lambda) = \text{true}] - \frac{1}{2}, \end{aligned}$$

where $\Pr [\cdot]$ is the probability function, which we will discuss in much more detail in Week 2b. For now, read $\Pr [\text{DL}^{\mathcal{A}}(\lambda) = \text{true}]$ as “the chance, over the random choices made inside the game, that the adversary wins.” Note that because an adversary that just picks its value randomly wins the $\text{DDH}^{\mathcal{A}}(\lambda)$ game half of the time, we define the advantage an adversary has to be how much better than probability $\frac{1}{2}$ it can do. Finally, we can make our assumptions formal and complete by saying that the DL assumption holds for a family of groups, $\mathbb{G}(\lambda)$, if for all programs, $\mathcal{A}$, the value $\text{Adv}_{\mathcal{A}}^{\text{dl}}(\lambda)$ is negligible. Similarly, we say that the CDH or DDH assumptions hold if $\text{Adv}_{\mathcal{A}}^{\text{cdh}}(\lambda)$ or $\text{Adv}_{\mathcal{A}}^{\text{ddh}}(\lambda)$ are negligible for all programs, $\mathcal{A}$. We will formalize what it means for a function of λ to be negligible next week, but for now just think of it as a value that becomes small exponentially fast as λ increases, or just think of

it informally as really really unbelievably small when key sizes are reasonably large.

Notice that an attack game turns the vague phrase “problem X is hard” into the precise phrase “every program has negligible advantage in the game $X^A(\lambda)$.” Crucially, this also gives our reductions a concrete target. When we say “if Y is easy then X is easy,” we now mean: “given a program $\mathcal{A}$ with non-negligible advantage in $Y^A(\lambda)$, we can build a program $\mathcal{A}'$ with non-negligible advantage in $X^A(\lambda)$.” The reduction is the construction of $\mathcal{A}'$ out of $\mathcal{A}$. The next exercise asks you to do this for the three assumptions above.

Exercise 4. *Use proof by reduction and the definitions above to informally show that*

(a) *If the DDH assumption holds, then the CDH assumption holds.*

(b) *If the CDH assumption holds, then the DL assumption holds.*

In order to solve this part, you may use two facts that we will prove later (though it is possible to do this exercise without):

(1) *If you have two independent events A and B , then $\Pr[A \text{ and } B] = \Pr[A] \cdot \Pr[B]$.*

(We will define independence and prove this in Week 2b.)

(2) *If $F(\lambda)$ is not negligible, then $F(\lambda)^2$ is also not negligible. (We will prove this next week.)*

(Hint: for each part, start by writing “suppose $\mathcal{A}$ is a program with non-negligible advantage in the easier-to-attack game,” then describe the program $\mathcal{A}'$ you build and which game it attacks. For (a), think about what an attacker that can compute Z lets you do in a game that only asks you to distinguish Z from random.)

Looking ahead

We have now seen how to phrase “hardness” as an attack game and how to relate one hardness assumption to another by reduction. Two things are still missing. First, all of

our advantages were written in terms of a probability function $\Pr[\cdot]$ that we have not yet defined. Second, our reductions so far have only needed to reason about probability very loosely (“non-negligible in, non-negligible out”); to prove that a concrete scheme like the one-time pad is secure, we will need to actually *compute* probabilities. Both gaps are filled in Week 2b, where we develop the small amount of probability theory we need and then use it, together with the reduction technique from this part, to prove our first real security theorems. We will also return to use the DL, CDH, and DDH assumptions above when we study Schnorr signatures next week.